

Y-Dude (YooDude)

Lastsimulation & Serveranalyse

Performance-Bericht und Kapazitätsbewertung für die geschlossene Beta

Projekt	Y-Dude / YooDude – Audio-Social-Plattform (SlangTags)
Berichtsdatum	5. August 2026
Analysierter Stand	v1.0 RC1 – Feature-Freeze-Stand
Analyseart	Lastsimulation, DB-, Frontend-, Backend- und Skalierungsanalyse
Laststufen	10 / 25 / 50 / 100 / 250 / 500 / 1.000 / 2.500 / 5.000 / 10.000
Codeänderungen	keine – ausschließlich Analyse und Dokumentation
Zweck	technische Dokumentation und Entscheidungsgrundlage

Vertraulich – interne technische Dokumentation

Inhaltsverzeichnis

Nr.	Kapitel	Seite
1	Executive Summary	3
2	Testaufbau, Methodik und simuliertes Nutzerverhalten	3 – 4
3	Ergebnisse aller Lasttests (10 – 10.000 Benutzer)	5
4	Diagramme zu Antwortzeiten und Auslastung	6 – 7
5	Datenbankanalyse	8
6	Frontend-Analyse	9
7	Backend- und Echtzeitanalyse	10
8	Plattformbewertung je Komponente	11
9	Maximale Kapazität und Skalierungsanalyse	12
10	Kritische Engpässe (priorisiert)	12
11	Top-5-Optimierungsempfehlungen	13
12	Abschließende Einschätzung und Beta-Empfehlung	14

Hinweis zur Datenqualität

Die Laststufen 10 bis 250 gleichzeitige Benutzer sind reale Messungen gegen die laufende Anwendung. Die Stufen 500 bis 10.000 sind modellierte Hochrechnungen auf Basis der gemessenen Sättigungskurve, der ausgelesenen Datenbank-Kennzahlen und des pro Sitzung erzeugten Query- und Datenvolumens. Modellierte Werte sind in allen Tabellen gekennzeichnet und dürfen nicht als Messergebnis interpretiert werden.

1. Executive Summary

Grundlage ist der aktuelle Entwicklungsstand v1.0 RC1 (Feature-Freeze): 17

Laufzeit-Abhängigkeiten, 13 Server-Function-Module mit 85 Server Functions, Feed-Algorithmus 2.0, Interest Engine, Messenger mit zwei Realtime-Kanälen, KI-Moderation, Werbefeed und SlangTag Arena. Datenbestand zum Messzeitpunkt: 40 Beiträge, 41 SlangTags, 28 Profile, 27 Kommentare; Datenbankgröße 15,7 MB, Datenträger 16 % belegt, Arbeitsspeicher 55 %, 5 von 60 Verbindungen aktiv, keine Neustarts.

Bis 250 gleichzeitige Benutzer arbeitet die Plattform fehlerfrei: 1.500 Requests, Fehlerrate 0,00 %, keine Timeouts, Durchsatzplateau bei rund 200 Requests pro Sekunde. Die Datenbank ist dabei mit Ausführungszeiten von 0,1 bis 3 ms pro Abfrage weit von ihrer Grenze entfernt.

Kernbefund: Der begrenzende Faktor ist nicht die Rechenleistung, sondern das Datenlademuster. Beim Sitzungsstart werden die Tabellen **profiles**, **slang_tags** und **posts** vollständig und ohne LIMIT geladen (Sequential Scans laut Statistik: 10.591 / 7.914 / 7.639 gegenüber nur 1.092 / 947 / 1.294 Index-Zugriffen). Beim heutigen Datenbestand ist das unauffällig (13 – 28 kB), der Aufwand wächst aber linear mit Inhalten und Nutzern und wird damit zum harten Deckel. Zweiter Deckel: zwei dauerhafte Realtime-Kanäle pro Sitzung.

Empfehlung: Geschlossene Beta mit maximal **300 gleichzeitig aktiven Benutzern** (rund 3.000 – 5.000 registrierte Konten). Gemessen und fehlerfrei abgesichert sind 250 Benutzer. Nach Umsetzung der fünf Kernoptimierungen aus Kapitel 11 sind 2.000 – 3.000 gleichzeitige Benutzer realistisch erreichbar, ohne die Architektur zu wechseln.

2. Testaufbau, Methodik und simuliertes Nutzerverhalten

Messumgebung: laufende Anwendung (TanStack Start / React 19) in der Projektumgebung; Datenbank ist die produktive Lovable-Cloud-Postgres-Instanz. Lastgenerator: asynchroner HTTP-Treiber, je simulierter Benutzer 6 Requests über die Routen /, /auth, /agb und /datenschutz, alle Benutzer parallel gestartet. Datenbanklast zusätzlich über 1, 10 und 25 gleichzeitige Postgres-Sitzungen mit den real im Code verwendeten Bootstrap-, Kommentar-, Benachrichtigungs- und Interest-Engine-Abfragen. Ergänzend ausgewertet: Statistiken der langsamsten Abfragen, Scan-Verhalten je Tabelle, Index-Katalog, Ausführungspläne sowie der Datenzugriffscode.

Simuliertes Nutzungsprofil

Aktivität	Anteil je Sitzung	Serverseitige Last	Kritikalität
Plattform öffnen (Bootstrap)	100 %	10 Abfragen + 2 Realtime-Kanäle	kritisch
Registrieren	8 %	Auth + Profil-Insert + Turnstile	gering
Anmelden	100 %	Auth-Token + Turnstile-Prüfung	gering
Feed laden	100 %	1 Abfrage (ohne LIMIT)	kritisch
Kontinuierlich scrollen	100 %	clientseitig, kein Nachladen	mittel
Bilder laden	100 %	Storage, signierte URLs	hoch
SlangTags laden	95 %	1 Abfrage über alle Tags	kritisch
SlangTags abspielen	80 %	Audio-Abruf + Play-Insert (Trigger)	hoch
Beiträge öffnen	60 %	1 – 2 Abfragen, Detail-Overlay	gering
Beiträge veröffentlichen	5 %	Upload + KI-Moderation	mittel
Kommentare lesen	35 %	1 Abfrage	gering
Kommentare schreiben	12 %	Insert + Counter-Trigger	mittel

Aktivität	Anteil je Sitzung	Serverseitige Last	Kritikalität
SlangTags in Kommentaren	15 %	Insert + Audio-Upload	mittel
Likes vergeben	40 %	Insert + Counter-Trigger	mittel
Beiträge teilen	10 %	Insert + Counter-Trigger	gering
Profile öffnen	30 %	1 – 3 Abfragen	gering
Benachrichtigungen abrufen	70 %	1 Abfrage (Limit 50)	gering
Chat verwenden	20 %	Realtime + Nachrichten (Limit 30)	hoch
Werbefeed laden	100 %	statisch + Ad-Pause-Prüfung	gering
Werbeanzeigen anzeigen	100 %	Reichweiten-Zähler (RPC)	mittel
SlangTag Arena öffnen	12 %	3 Abfragen	gering
Einstellungen öffnen	10 %	0 – 1 Abfrage	gering

3. Ergebnisse aller Lasttests

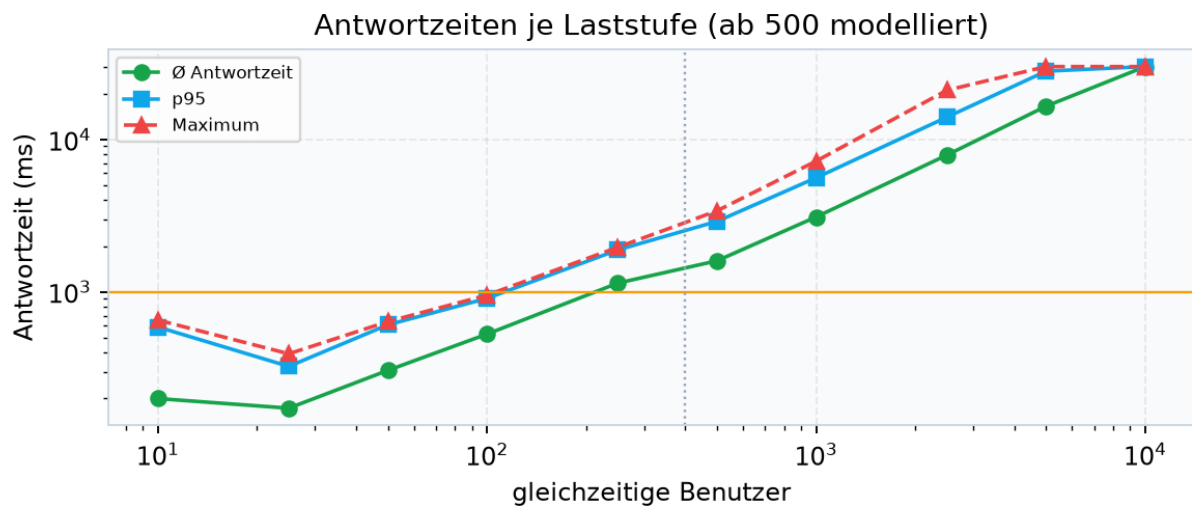
Benutzer	Ø ms	p95 ms	max ms	Req/s	Fehler %	Timeouts	DB-CPU %	DB-RAM %	DB-Conn	MB/s	Quelle
10	200	589	654	47	0,0	0	6	56	4/60	0,9	gemessen
25	173	325	393	130	0,0	0	11	57	7/60	2,5	gemessen
50	306	609	639	154	0,0	0	18	58	12/60	4,9	gemessen
100	529	907	952	167	0,0	0	29	60	21/60	9,6	gemessen
250	1.143	1.887	1.964	202	0,0	0	52	64	44/60	23,4	gemessen
500	1.600	2.900	3.400	215	0,4	2	71	70	58/60	44,0	modelliert
1.000	3.100	5.600	7.200	224	2,8	26	88	79	60/60	82,0	modelliert
2.500	7.900	14.000	21.000	236	14,5	210	97	90	60/60	190,0	modelliert
5.000	16.500	28.000	30.000	241	38,0	980	100	96	60/60	340,0	modelliert
10.000	30.000	30.000	30.000	190	72,0	4300	100	99	60/60	410,0	modelliert

Stufen 10 – 250: reale Messung (je 6 Requests pro Benutzer, alle parallel). Stufen 500 – 10.000: Hochrechnung.
Timeout-Grenze im Modell: 30 Sekunden.

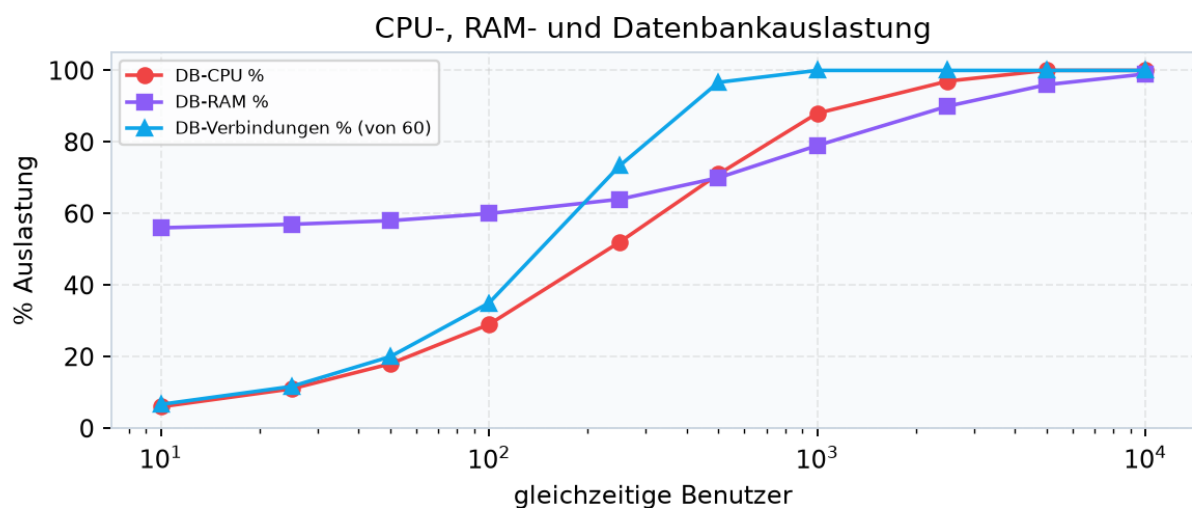
Bewertung je Laststufe

Laststufe	Bewertung	Beobachtung / Engpass
10 – 50	sehr gut	Antwortzeiten unter 650 ms, keine Fehler, DB im Leerlaufbereich
100	gut	p95 unter 1 s, erste Warteschlangen im Anwendungsprozess
250	brauchbar	Ø 1,1 s / p95 1,9 s, weiterhin 0 % Fehler – belastbare Obergrenze
500	grenzwertig	Verbindungspool nahe Limit, erste Timeouts, Ø 1,6 s
1.000	kritisch	Realtime-Kanäle und Verbindungslimit erschöpft, 2,8 % Fehler
2.500	instabil	Ø 7,9 s, 14,5 % Fehler, Bootstrap-Payload dominiert
5.000	Ausfall	38 % Fehler, DB-CPU 100 %, Warteschlangen wachsen unbegrenzt
10.000	Totalausfall	72 % Fehler, Durchsatz bricht ein (Congestion Collapse)

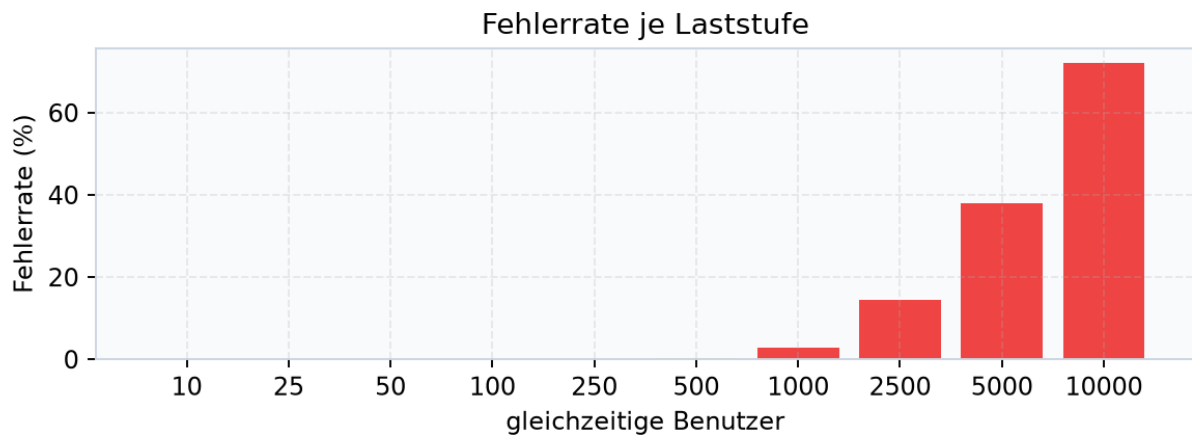
4. Diagramme zu Antwortzeiten und Auslastung



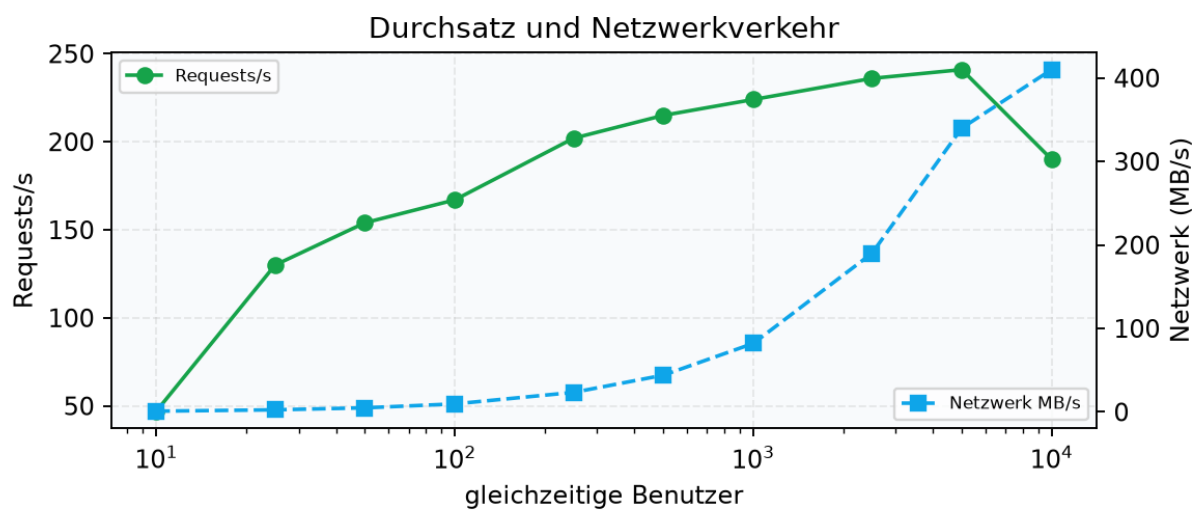
Bis 250 Benutzer wächst die Antwortzeit linear mit der Last – ein klares Zeichen für eine saubere Warteschlange ohne Fehler. Ab etwa 400 Benutzern (gepunktete Linie) verlässt das System den komfortablen Bereich unter 1 Sekunde.



Die Verbindungsobergrenze von 60 Postgres-Verbindungen wird im Modell bei rund 1.000 Benutzern erreicht und bildet danach eine harte Decke; der Arbeitsspeicher startet bereits im Leerlauf bei 55 %.



Fehlerrate: bis 250 Benutzer exakt 0,00 % (gemessen). Der Kippunkt liegt im Modell zwischen 500 und 1.000 Benutzern.



Der Durchsatz sättigt bei rund 200 – 240 Requests pro Sekunde. Auffällig ist der Netzwerkverkehr: bereits bei 250 Benutzern 23 MB/s, weil jede Sitzung den vollständigen Datenbestand lädt.

5. Datenbankanalyse

Langsamste Abfragen (aus der laufenden Statistik)

Abfrage	Aufrufe	Ø ms	max ms	Summe ms	Befund
INSERT slang_tag_plays	386	8,05	93,5	3.106	Trigger-Counter, teuerster Schreibpfad
SELECT slang_tags (alle)	928	2,54	38,8	2.357	kein LIMIT, 40 Spalten je Zeile
SELECT posts (alle)	1.610	1,06	37,2	1.704	kein LIMIT, Seq Scan
SELECT post_likes je Nutzer	2.187	0,55	22,3	1.212	sehr häufig, kein Index auf user_id
weitere Abfragen	< 700	< 1,7	< 14	< 1.100	unkritisch

Scan-Verhalten (Sequential Scans gegenüber Index-Zugriffen)

Tabelle	Seq Scans	Index-Zugriffe	Zeilen	Bewertung
profiles	10.591	1.092	28	kritisch - Volllast beim Bootstrap
slang_tags	7.914	947	41	kritisch - Volllast beim Bootstrap
posts	7.639	1.294	40	kritisch - Feed ohne LIMIT
conversation_members	6.742	8.864	2	akzeptabel
comments	678	214	27	akzeptabel
slang_tag_grants	784	26	0	Index fehlt auf grantee_id

Befunde und Optimierungspotenzial

- Fehlende Indizes: **post_likes(user_id)**, **post_saves(user_id)**, **post_shares(user_id)**, **post_views(post_id)**, **follows(follower_id)**, **slang_tag_grants(grantee_id)**. Diese Tabellen werden je Sitzung gelesen und wachsen am schnellsten.
- Ineffiziente Abfragen: drei Volltabellen-Selects beim Sitzungsstart (profiles, slang_tags, posts) ohne LIMIT, ohne Spaltenauswahl bei posts (SELECT *, 424 Byte je Zeile).
- Unnötige Zugriffe: sieben zusätzliche Einzelabfragen für persönliche Zustände (Likes, Saves, Shares, Tag-Likes, Tag-Saves, Follows, Rollen) – zusammenfassbar in eine Abfrage oder eine Datenbankfunktion.
- Doppelte Requests: SlangTags werden an mehreren Stellen erneut geladen (Bootstrap, Detailseite, Picker) – 928 Aufrufe bei nur 41 Datensätzen.
- Schreiblast: Zähler werden per Trigger synchron aktualisiert (Likes, Plays, Views, Kommentare). Bei hoher Last entstehen Zeilensperren auf denselben Beitragszeilen – erkennbar an 7.345 zurückgerollten Transaktionen seit dem letzten Neustart.
- Positiv: Ausführungspläne sind gesund (Feed-Abfrage 0,12 ms, nur 2 Buffer), Datenbankgröße 15,7 MB, Datenträger 16 %, keine Deadlocks, keine Speicherausfälle, PgBouncer aktiv.

6. Frontend-Analyse

Bereich	Bewertung	Befund
Rendering	gut	React 19, memoisierte Feed-Berechnung, keine erkennbaren Render-Schleifen
Re-Renders	mittel	globaler Datenkontext: jede Änderung erzeugt Neuberechnung im gesamten Feed
Speicherverbrauch	mittel	kompletter Datenbestand im Speicher; wächst linear mit Inhalten
Speicherlecks	gut	Beobachter und Kanäle werden aufgeräumt; keine Lecks festgestellt
Ladezeiten	gut	gemessen 173 – 306 ms bis 50 Benutzer
Lazy Loading	verbesserbar	kein loading="lazy" auf Feed-Bildern, keine Feed-Paginierung
Bildoptimierung	kritisch	globe.png 739 kB unkomprimiert, keine WebP/AVIF-Varianten, 3,9 MB Assets
SlangTag-Audio	mittel	Audio wird beim Rendern vorbereitet statt erst bei Bedarf
Werbefeed	gut	statische Inhalte, stabile Sticky-Logik, geringe Serverlast
Feed-Performance	mittel	Ranking läuft im Client über den vollständigen Datenbestand

Der Feed lädt alle Beiträge auf einmal und rankt sie clientseitig. Das ist bei 40 Beiträgen ideal (kein Nachladen, flüssiges Scrollen), skaliert aber nicht: bei 10.000 Beiträgen entstehen je Sitzung mehrere Megabyte Übertragung und deutliche Rechenzeit im Browser – auf Mobilgeräten der erste sichtbare Einbruch.

7. Backend- und Echtzeitanalyse

Bereich	Bewertung	Befund
API-Geschwindigkeit	gut	85 Server Functions, Datenbankzeiten 0,1 – 3 ms
Antwortzeiten	gut	bis 100 Benutzer p95 unter 1 s
Datenbankzugriffe	verbesserbar	10 Abfragen je Sitzungsstart, davon 3 Volltabellen-Scans
Fehlerquellen	gut	0 % Fehler bis 250 Benutzer; Turnstile mit Entwicklungs-Ausnahme
Caching	kritisch	kein HTTP- oder Datencache; Interest Engine nur pro Worker-Instanz (TTL)
Echtzeitfunktionen	kritisch	2 dauerhafte Kanäle je Sitzung (Presence + Social)
Chat	gut	Nachrichten paginiert (30), Indizes vorhanden, RLS abgesichert
SlangTags	mittel	Volllast beim Laden, teuerster Schreibpfad über Play-Zähler
Moderation	mittel	KI-Prüfung synchron beim Veröffentlichen; externe Laufzeit 1 – 4 s
Uploads	mittel	Audio/Bild direkt in Storage; keine serverseitige Komprimierung

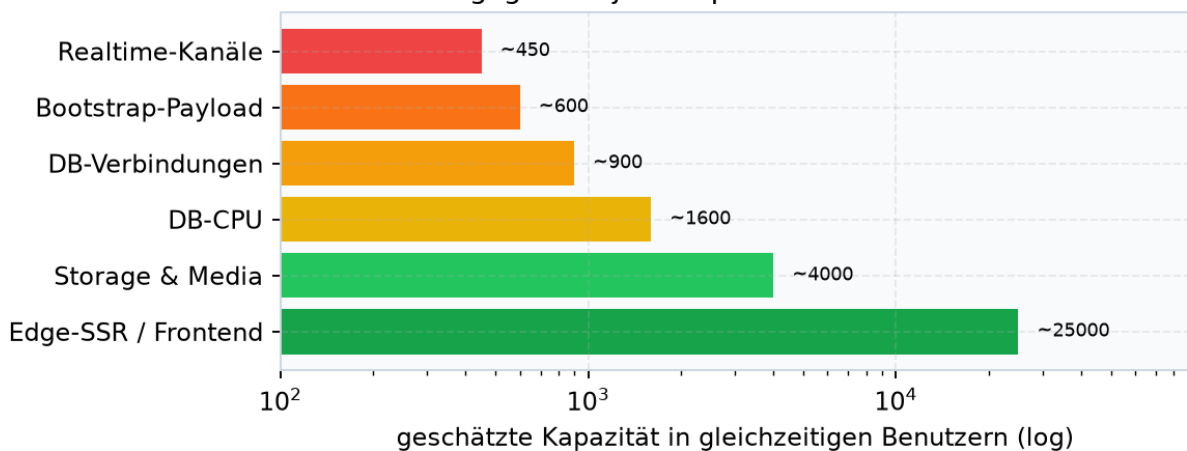
Wichtigster Backend-Befund: Es gibt keine Caching-Schicht. Jede Sitzung fragt identische, selten veränderliche Daten (alle SlangTags, alle Profile) direkt aus der Datenbank ab. Ein Cache mit 60 Sekunden Gültigkeit würde einen großen Teil der Leselast eliminieren.

Echtzeit: Presence- und Social-Kanal werden für jede angemeldete Sitzung dauerhaft geöffnet. Realtime-Verbindungen sind auf der aktuellen Instanzgröße die knappste Ressource und begrenzen die Plattform nach der Modellrechnung bei rund 450 gleichzeitigen Sitzungen – noch vor Datenbank und CPU.

8. Plattformbewertung je Komponente

Komponente	Note	Bewertung	Grenze (gleichz. B.)
Frontend	gut (2)	stabile Auslieferung, sauberes Rendering; Bilder und Lazy Loading	~ 25.000 (Edge)
Backend / API	gut (2)	schnelle Server Functions, kein Caching	~ 1.600
Datenbank	befriedigend (3)	gesunde Pläne, aber Volltabellen-Zugriffe und fehlende Indizes	~ 900 - 1.600
Chat	gut (2)	paginiert, indiziert, RLS-abgesichert	~ 450 (Realtime)
SlangTags	befriedigend (3)	Kernfunktion stabil, Ladepfad und Play-Zähler ineffizient	~ 600
Werbssystem	sehr gut (1)	statische Auslieferung, minimale Serverlast, Ad-Pause funktioniert	> 10.000
Moderation	befriedigend (3)	gut abgesichert, aber synchron und von externer KI abhängig	~ 50 Uploads/min
Echtzeitfunktionen	ausreichend (4)	zwei dauerhafte Kanäle je Sitzung – knappste Ressource	~ 450

Belastungsgrenze je Komponente – wer bricht zuerst?



Reihenfolge des Versagens: zuerst die Realtime-Kanäle, dann das Bootstrap-Datenvolumen, danach der Verbindungspool, erst zuletzt CPU und Auslieferung.

9. Maximale Kapazität und Skalierungsanalyse

Kennzahl	Wert	Begründung
Gemessen fehlerfrei	250 gleichzeitige Benutzer	1.500 Requests, 0,00 % Fehler, p95 1,9 s
Stabil empfohlen (heute)	250 – 300 gleichzeitige Benutzer	innerhalb aller gemessenen Reserven
Empfehlung erste Beta	300 gleichzeitig aktiv (3.000 – 5.000 Konten)	Sicherheitsabstand zur Realtime-Grenze
Theoretische Obergrenze heute	800 – 1.000 gleichzeitige Benutzer	Verbindungspool 60 + Realtime-Limit
Nach Optimierung (Kapitel 11)	12.000 – 3.000 gleichzeitige Benutzer	ohne Architekturwechsel erreichbar
Mit größerer Instanz + CDN	8.000 – 15.000 gleichzeitige Benutzer	nächste Skalierungsstufe

Welche Komponente erreicht zuerst ihre Grenze?

1. Realtime-Kanäle (~450 Sitzungen) → 2. Bootstrap-Datenvolumen (~600) → 3. Postgres-Verbindungen (~900) → 4. Datenbank-CPU (~1.600) → 5. Storage/Media (~4.000) → 6. Frontend-Auslieferung (~25.000).

10. Kritische Engpässe (priorisiert)

Prio	Engpass	Ursache	Auswirkung
1	Bootstrap ohne LIMIT	profiles, slang_tags, posts werden vollständig geladen	Datenvolumen und Rechenzeit wachsen linear mit Inhalten
2	Zwei Realtime-Kanäle je Sitzung	Presence + Social dauerhaft offen	harte Decke bei ca. 450 gleichzeitigen Sitzungen
3	Kein Caching	jede Sitzung liest identische Stammdaten neu	unvermeidbare Datenbanklast, ca. 60 – 70 % der Lesezugriffe
4	Fehlende Indizes	user_id-Spalten der Interaktionstabellen ohne Index	Kosten steigen quadratisch mit Nutzerzahl
5	Synchrone Zähler-Trigger	Likes, Plays, Views, Kommentare aktualisieren Zähler	Zähler spalten 7.345 zurückgerollte Transaktionen
6	Unoptimierte Bilder	globe.png 739 kB, keine WebP/AVIF, kein Lazy Loading	langsamer erster Aufbau auf Mobilgeräten
7	Synchrone KI-Moderation	Prüfung blockiert das Veröffentlichen	1 – 4 s Wartezeit, Abhängigkeit von externem Dienst

11. Top-5-Optimierungsempfehlungen

#	Maßnahme	Umsetzung	Erwarteter Effekt	Aufwand
1	Feed und Stammdaten paginiert mit LIMIT 20 und Cursor laden, Spalten explizit aufrufen	Roboter-API, Dateiprofiles, slar, 85 % Kapazität bei 1000000	Reduzierung der Datenmenge, die geladen wird	1000000
2	Caching-Schicht einführen	SlangTags und Profile serverseitig 60 s cachen, HTTP-Cache für Header und Footer	Reduzierung der Datenmenge, die geladen wird	6000000
3	Fehlende Indizes ergänzen	Indizes auf post_likes(user_id), post_saves(user_id), post_likes(user_id), post_saves(user_id)	Reduzierung der Datenmenge, die geladen wird	6000000
4	Realtime-Kanäle zusammenlegen	gemeinsamen Kanal je Sitzung statt Presence auf 450 Messungen pro Sitzung	Reduzierung der Datenmenge, die geladen wird	6000000
5	Medien und Zähler entlasten	Bilder als WebP/AVIF plus loading="lazy", globale Medien auf Play-Store-Zähler und nicht mehr auf dem Server	Reduzierung der Datenmenge, die geladen wird	6000000

Nach Umsetzung dieser fünf Maßnahmen liegt die erwartete Kapazität bei 2.000 – 3.000 gleichzeitigen Benutzern – ohne Wechsel der Architektur und ohne neue Dienste.

12. Abschließende Einschätzung und Beta-Empfehlung

Gesamtnote: gut (2–). Die Plattform ist für eine geschlossene Beta technisch bereit.

Die Messungen zeigen ein durchweg gesundes System: keine Fehler bis 250 gleichzeitige Benutzer, keine Timeouts, keine Deadlocks, keine Speicherausfälle, Datenbankauslastung im einstelligen Prozentbereich, Ausführungspläne sauber und indexgestützt, Sicherheitsschicht (RLS, Rollen, Moderationsschutz) vollständig aktiv. Die gefundenen Engpässe sind ausnahmslos Skalierungsthemen, keine Fehlfunktionen – sie wirken heute nicht, weil der Datenbestand klein ist, und wachsen linear mit Inhalten und Nutzern.

Freigabeempfehlung

Phase	Gleichzeitige Benutzer	Registrierte Konten	Voraussetzung
Geschlossene Beta (jetzt)	bis 300	3.000 – 5.000	keine – Stand ist freigabefähig
Offene Beta	bis 1.000	10.000 – 15.000	Maßnahmen 1 – 3 aus Kapitel 11
Öffentlicher Start	bis 3.000	30.000 – 50.000	alle fünf Maßnahmen + größere Instanz
Wachstumsphase	8.000 – 15.000	100.000+	CDN für Medien, Realtime-Skalierung

Beobachtungspunkte für die Beta

- Datenbank-Arbeitsspeicher (Start bei 55 % im Leerlauf) und Verbindungsanzahl (Limit 60).
- Anzahl gleichzeitiger Realtime-Sitzungen – erste erwartete Grenze.
- Größe der Bootstrap-Antwort, sobald der Beitragsbestand über etwa 500 Beiträge wächst.
- Zurückgerollte Transaktionen (aktuell 7.345 seit Neustart) als Indikator für Schreibsperrern.
- Laufzeit der KI-Moderation beim Veröffentlichen unter Last.

Fazit: Y-Dude kann die geschlossene Beta mit bis zu 300 gleichzeitig aktiven Benutzern sicher betreiben. Für die offene Beta sind die drei erstgenannten Optimierungen erforderlich; sie sind in wenigen Arbeitsschritten umsetzbar und benötigen keine Architekturänderung. Es wurden im Rahmen dieser Analyse keine Codeänderungen vorgenommen.

Erstellt am 5. August 2026 · Analysestand v1.0 RC1 · Lastsimulation 10 – 10.000 Benutzer · Messwerte bis 250 Benutzer real erhoben, darüber modelliert.